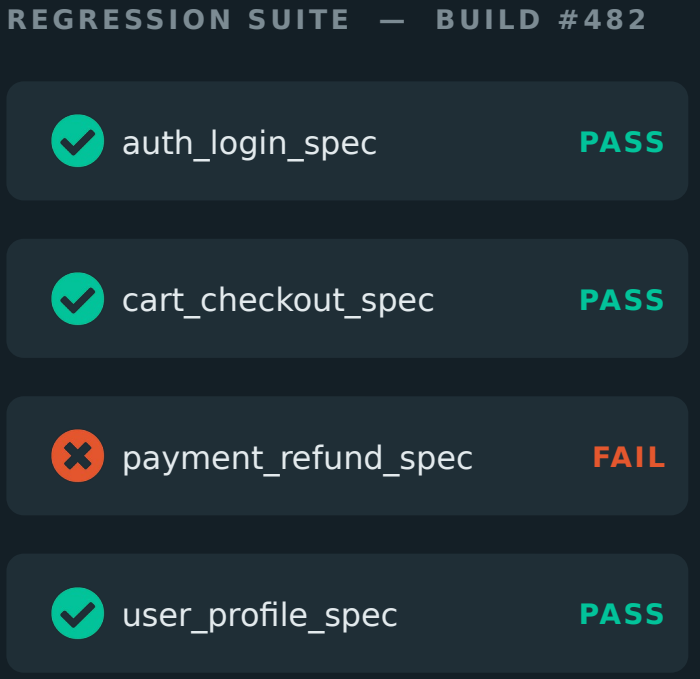
QUALITY ENGINEERING · FUNDAMENTALS

Regression Testing

Making sure new code does not break what already works.

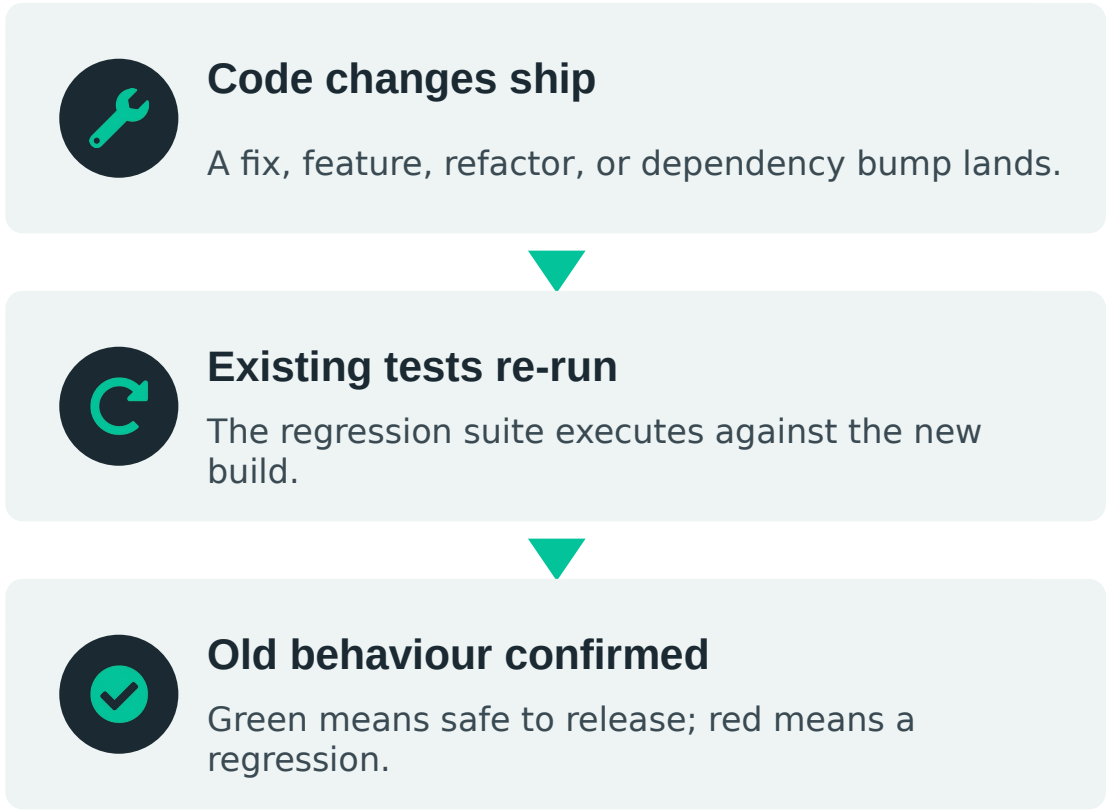
Selenium · Playwright · CI pipelines



What regression testing is

A quality check that re-runs existing test cases after the code changes — an update, a bug fix, or a new feature — to confirm that software which already worked still works.

- Verifies existing behaviour, not new behaviour
- Triggered by every meaningful code change
- Almost always automated in practice



Why it matters

Small changes cast long shadows across a shared codebase.

Catches unintended bugs

Hidden side effects in shared code and libraries break features nobody edited.

Protects core business workflows

Login, search, checkout and payments keep working for the users who pay you.

Builds release confidence

Teams ship frequently because a green suite is evidence rather than optimism.

Three common approaches

Choose by risk, time budget, and how much of the system the change touches.

Retest-all

Run every case in the suite. Maximum safety at maximum cost — kept for major releases and risky rewrites.

Coverage: total · Cost: highest

Selective

Run only the tests mapped to the modified modules and whatever depends on them. Fast feedback for routine work.

Coverage: targeted · Cost: low

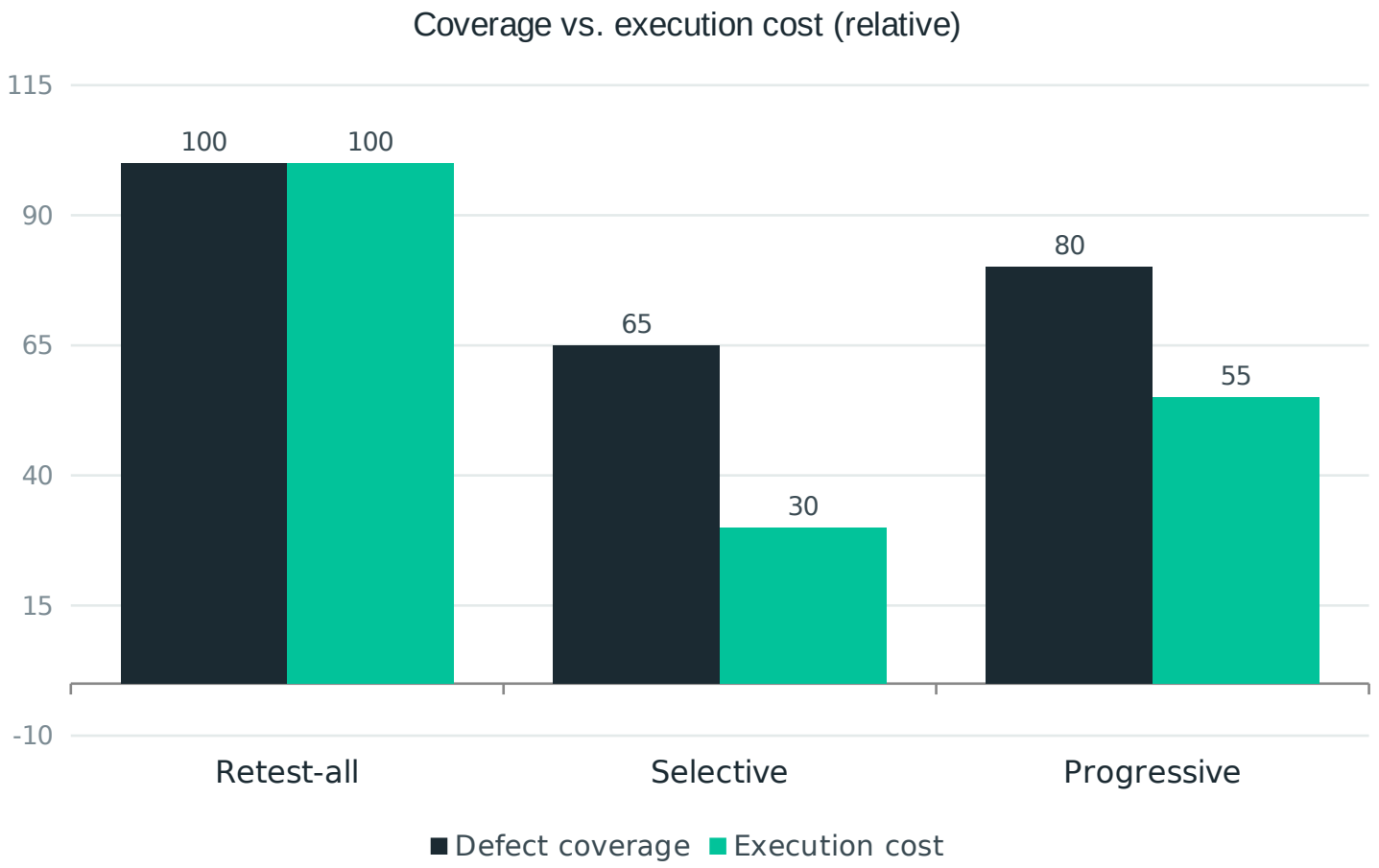
Progressive

Write tests for the new feature and check that it integrates without disturbing established behaviour.

Coverage: new + old · Cost: medium

Selective runs depend on a trustworthy map of which tests cover which code — without it, you are guessing.

Choosing an approach



Rules of thumb

- Small, isolated fix → selective run.
- Shared library, config or schema change → retest-all.
- New feature → progressive, plus a targeted regression pass.
- Before any major release → retest-all, no exceptions.

Illustrative relative values — real numbers depend on your suite size and codebase.

How a regression cycle runs

Five steps that repeat on every build, whether a human or a pipeline drives them.

1

Detect the change

Identify exactly what moved: code, config, or dependencies.

2

Select the scope

Decide between retest-all, selective, or progressive.

3

Run the suite

Execute in CI, in parallel, on a production-like build.

4

Triage results

Separate real regressions from flaky or outdated tests.

5

Fix and maintain

Repair the code, update the suite, retire dead cases.

A failing test is either a real regression or a maintenance debt. Both need an owner — neither should be muted.

When to run it

Any change that could ripple beyond the module it lives in.

Bug fixes

The fix itself, plus everything that shared the faulty code path.

New features

Confirm the addition integrates without disturbing existing flows.

Updates and config changes

Library upgrades, API versions, environment and infrastructure edits.

Data and performance work

Schema migrations, query tuning and caching touch many features at once.

Tools and automation

Selenium

Long-established browser automation with broad language and browser support, and a very large ecosystem.

Playwright

Modern cross-browser automation with auto-waiting, parallel execution, and built-in tracing.

Manual regression testing stops scaling after a handful of releases.

Make the suite worth trusting

- Automate the highest-value paths first — login, checkout, payments.
- Keep tests independent and repeatable, each owning its own data.
- Run the suite on every merge in CI, not the night before a release.
- Quarantine and fix flaky tests; a suite nobody trusts gets ignored.

TAKEAWAYS

What to remember

- New code is guilty until the existing tests prove otherwise.
- Scope the run to the risk: selective for small fixes, retest-all before releases.
- Automate it, run it in CI, and treat the suite as production code.
- Every failure is information — triage it, never mute it.

Sources: GeeksforGeeks · TechTarget · Opkey · TestingXperts